লুপ দিয়ে আঁকা: নেস্টেড for

পাঠ 13 — 1IOT, SST-ICT-01 — অনুশীলন: তারকাচিহ্ন, ভরা ও ফাঁপা আয়তক্ষেত্র, ত্রিভুজ

20/10/2026

1 আজ আমরা কী করলাম

আজ আমরা পাঠ 12-এর গঠনগুলো (**for**, **while**, **do while**) একটি **অনুশীলনের ধারায়** ব্যবহার করলাম, প্রতিটি আগেরটির চেয়ে একটু কঠিন: কোনোটিই আগেরটির চেয়ে **সহজ** নয়। শুরু এক সারি তারকাচিহ্ন দিয়ে, তারপর একটি **আয়তক্ষেত্র**, তারপর একটি **ফাঁপা** আয়তক্ষেত্র, শেষে একটি **ত্রিভুজ**। সব অনুশীলনে ব্যবহারকারীর দেওয়া **ডেটা যাচাইয়ের** সমস্যাও আছে।

- প্রতিটি অনুশীলন onlinegdb-তে একটি **নতুন প্রজেক্টে** লেখা হয়: আগে করা প্রজেক্ট **বদলানো হয় না**, রাখা হয়।
- এগোনোর আগে যা লেখা হয়েছে তা **পরীক্ষা করো**, প্রোগ্রাম শেষ না হলেও: এতে কোড ছোট থাকতেই ভুল ধরা পড়ে।
- কোড সবসময় **ইন্ডেন্ট করতে** হবে (অংশ 4.4)।

টীকা

কোন অক্ষর ছাপানো হচ্ছে তার গুরুত্ব নেই: **তারকাচিহ্ন** (**asterisco**) ঐতিহ্য অনুযায়ী ব্যবহার হয়, কিন্তু `x`, `@` বা অন্য যেকোনো অক্ষরও চলে।

2 অনুশীলন 1: এক সারি তারকাচিহ্ন

উদাহরণ

প্রশ্ন। একটি প্রোগ্রাম লেখো যা ব্যবহারকারীর কাছে **0-এর বড়** একটি সংখ্যা চায়। তারপর প্রোগ্রাম সেই সংখ্যার সমান **তারকাচিহ্ন** ছাপায়।

2.1 ডেটা যাচাই করতে হবে

প্রোগ্রাম **চলার সময়** (**a runtime**) যন্ত্রে ডেটা ঢোকাতে সবসময়ের মতো লাগে একটি `printf`, যা ব্যবহারকারীকে বলে আমরা কী চাই, আর একটি `scanf`, যা ডেটা **নেয়**।

নতুন বিষয় হলো সংখ্যাটি **0-এর বড়** হতে হবে: 1, 2, 3, 4… চলে, 0, -1, -5 **চলে না**। ব্যবহারকারী শর্ত না মানা ডেটা দিলে যন্ত্র **মেনে নেয় না** এবং **আবার প্রশ্ন করে**।

সবচেয়ে সুবিধাজনক গঠন কোনটি? **do while**, কারণ প্রশ্ন **অন্তত একবার** করতে হবে। `while`-এর শর্ত বলে **কখন পুনরাবৃত্তি**, অর্থাৎ কখন ডেটা **ভুল**: `N < 1` যতক্ষণ, পুনরাবৃত্তি।

> **সতর্কতা**
>
> `N < 0` নয়, `N < 1` কেন? `N < 0` হলে **শূন্য** মেনে নেওয়া হতো (0, 0-এর ছোট নয়), কিন্তু আমরা **শূন্য চাই না**। সঠিক লেখা দুটি, **সমতুল্য**: `N < 1` অথবা `N <= 0`। একটি বেছে নাও।

শেষের `while` বন্ধ বন্ধনীর একই লাইনে বা নতুন লাইনে থাকতে পারে: কীভাবে নতুন লাইনে যাও বা ইন্ডেন্ট করো তা C-র কাছে **গুরুত্বপূর্ণ নয়**, প্রোগ্রাম একইভাবে চলে। সব এক লাইনে লেখা যেত, কিন্তু তখন **পড়া যায় না**। `do`-এর বন্ধনীর ভেতরে সবসময় **ইন্ডেন্টেশনের ফাঁক** মনে রেখো।

2.2 প্রথম অংশ সঙ্গে সঙ্গে পরীক্ষা করি

তারকাচিহ্নের লুপ লেখার আগে যা আছে তা চালিয়ে (**Run**) দেখা ভালো **ভুল আছে কিনা**। প্রোগ্রাম সংখ্যা চায়; অবশ্যই এখনো তারকাচিহ্ন ছাপে না, কিন্তু প্রশ্নটি পরীক্ষা করা যায় (**Inserisci il numero di asterischi da stampare** = ছাপানোর তারকাচিহ্নের সংখ্যা দাও):

```
Inserisci il numero di asterischi da stampare: 0
Inserisci il numero di asterischi da stampare: -5
Inserisci il numero di asterischi da stampare: 5
```

0 ও -5 দিলে প্রশ্ন **আবার হয়**: ডেটা মানা হয় না। 5 দিলে প্রোগ্রাম **লুপ থেকে বেরিয়ে** শেষে পৌঁছায়। এ পর্যন্ত ঠিক।

2.3 তারকাচিহ্ন: for

এবার `N`টি তারকাচিহ্ন ছাপাতে হবে। পুনরাবৃত্তির সংখ্যা **আগেই জানি** (`N`), তাই **for** ব্যবহার হয়। গোনার জন্য একটি **নতুন ভেরিয়েবল** লাগে: `N` **হতে পারে না**, কারণ `N` আমরা **সীমা** (কতদূর যাব) হিসেবে ব্যবহার করছি। নাম দিই `x`: 0 থেকে শুরু, `x < N` যতক্ষণ পুনরাবৃত্তি, প্রতি ঘূর্ণনে `x++`।

লুপের ভেরিয়েবল **সরাসরি `for`-এর ভেতরে** ঘোষণা করা যায়, ক্লাসে যেমন করি: `for (int x = 0; x < N; x++)`। এতে `x` **শুধু লুপের ভেতরে** থাকে, প্রোগ্রামের শুরুতে ঘোষণা করতে হয় না।

তবে এভাবে লেখা `for` **এখনো কিছু করে না**: কতবার পুনরাবৃত্তি হবে বলেছি, কিন্তু **কী** পুনরাবৃত্তি হবে বলিনি। ভেতরে যায় একটি তারকাচিহ্ন ছাপানো, `printf("*");`।

```c
#include <stdio.h>

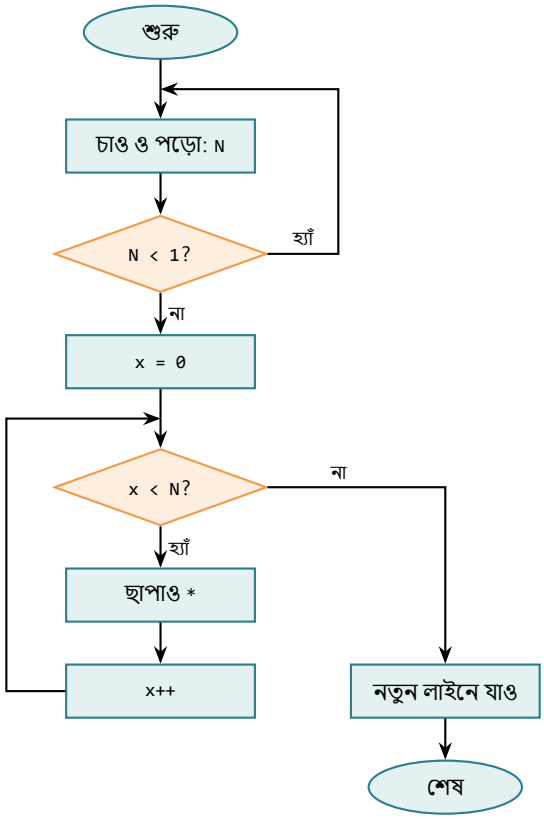
int main()
{
    int N;

    // সংখ্যা 0-এর বড় না হওয়া পর্যন্ত চাই
    do
    {
        printf("Inserisci il numero di asterischi da stampare: ");
        scanf("%d", &N);
    } while (N < 1);

    // একই সারিতে Nটি তারকাচিহ্ন ছাপাই
    for (int x = 0; x < N; x++)
    {
        printf("*");
    }
    // শেষে নতুন লাইন (পাঠ্যে যোগ করা: কার্সর সারিতে থাকে না)
    printf("\n");

    return 0;
}
```

```
Inserisci il numero di asterischi da stampare: 5
*****
```



ছবি/নকশা 1: অনুশীলন 1: ডেটা যাচাইয়ের `do while`, তারপর তারকাচিহ্নের `for`।

2.4 পুনরালোচনা: do while ও while

do while হলো `while`-এর একটি রূপ, যা নিশ্চিত করে বন্ধনীর ভেতরের কোড **অন্তত একবার** চলবে। `while` আবার **আগে** শর্ত যাচাই করে: প্রথমবারেই মিথ্যা হলে কোড **একবারও** চলে না, তাই ব্যবহারকারীকে প্রশ্ন কখনো করা হতো না। তাই ডেটা চাইতে `do while` ব্যবহার হয়।

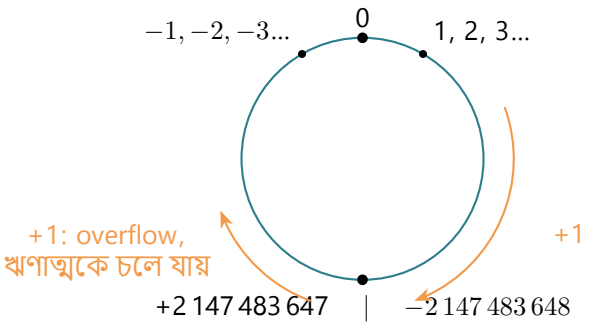
2.5 ক্লাসের একটি প্রশ্ন: সংখ্যা খুব বড় হলে?

খুব বড় সংখ্যা দিয়ে প্রোগ্রাম চালালে অদ্ভুত কিছু ঘটে: কখনো প্রোগ্রাম এমনভাবে আবার প্রশ্ন করে যেন সংখ্যাটি ঋণাত্মক, কখনো **না থেমে** তারকাচিহ্ন ছাপতে থাকে। কারণ হলো **ডেটা টাইপ**।

একটি `int`-এর একটি **বিটের আকার** আছে: C **অন্তত 16 বিট** নিশ্চিত করে, তবে আসল আকার যন্ত্রের ওপর নির্ভর করে। আজকের পিসিতে (ও onlinegdb-তে) `int` হলো **32 বিট**। 32 বিটে সংমিশ্রণ 2^{32}, প্রায় **4 বিলিয়ন** (4 294 967 296)। `int`-এ **ধনাত্মক ও ঋণাত্মক** সংখ্যা থাকে বলে এই পরিসর অর্ধেকে ভাগ হয়: প্রায় -2 বিলিয়ন থেকে প্রায় $+2$ বিলিয়ন (সঠিকভাবে $-2\,147\,483\,648$ থেকে $+2\,147\,483\,647$)।

সংজ্ঞা

কম্পিউটারে পূর্ণসংখ্যাগুলো একটি **বৃত্তের** মতো সাজানো: 0 থেকে শুরু, 1, 2, 3... বাড়তে বাড়তে বৃত্তের অর্ধেকে সর্বোচ্চ, প্রায় **+2 বিলিয়ন**। অন্য দিকে ঋণাত্মক, -1, -2, -3... প্রায় -2 **বিলিয়ন** পর্যন্ত। সবচেয়ে বড় সংখ্যায় 1 যোগ করলে আরও বড় সংখ্যা **পাই না**: চলে যাই -2 বিলিয়নে। গণিতের দিক থেকে এটি ভুল: এটি **overflow**।



একসময় overflow এমন সমস্যা ছিল যাতে **সবসময় ধাক্কা** লাগত; আজকের যন্ত্রে সেখানে পৌঁছানো কঠিন, কিন্তু **অসম্ভব নয়**। আমাদের প্রোগ্রামে:

- **3 বিলিয়ন** দিলে `int`-এ থাকে একটি **ঋণাত্মক** সংখ্যা ($-1\,294\,967\,296$): `do while` এটিকে ভুল ধরে **আবার প্রশ্ন করে**;
- **5 বিলিয়ন** দিলে `int`-এ থাকে একটি **ধনাত্মক** সংখ্যা (705 032 704): প্রোগ্রাম মেনে নিয়ে **700 মিলিয়নের বেশি** তারকাচিহ্ন ছাপে, যেন আর থামে না।

টীকা

গভীরে। ক্লাসে দেখেছি, 2 বিলিয়নের বেশি সংখ্যায় প্রোগ্রাম আবার প্রশ্ন করে, যেন সংখ্যাটি ঋণাত্মক। ওপরের দুটি ক্ষেত্র (3 ও 5 বিলিয়ন) পাঠের পরে পিসিতে পরীক্ষা করা হয়েছে, "আর থামে না" ক্ষেত্রটিও ব্যাখ্যা করতে। C মান অনুযায়ী, চিহ্নযুক্ত `int`-এর সীমা পেরোনো **অনির্ধারিত আচরণ**: "বৃত্তে ঘোরা" বাস্তবে আমাদের কম্পিউটারে যা ঘটে। সঠিক মান মনে রাখার দরকার নেই: জরুরি হলো জানা যে প্রতিটি **ডেটা টাইপের** একটি **সীমা** আছে।

2.6 একটি অক্ষর লিখলে?

`scanf`-এ আমরা লিখেছি `%d`: তাকে একটি **পূর্ণসংখ্যা** নিতে বলেছি। ব্যবহারকারী একটি **অক্ষর** (বা নিজের নাম) লিখলে `scanf` তা **পড়তে পারে না**, ভেরিয়েবল বৈধ মান পায় না এবং অক্ষরটি না পড়া অবস্থায় থেকে যায়: প্রোগ্রাম **অসীমবার** প্রশ্ন করতে পারে বা অদ্ভুত আচরণ করতে পারে। আপাতত আমরা **শুধু সংখ্যা** নিতে জানি; **শব্দ** নিতে যে সরঞ্জাম লাগে তা পরে, মোটামুটি **নভেম্বরের** দিকে দেখব।

3 অনুশীলন 2: তারকাচিহ্নের আয়তক্ষেত্র

উদাহরণ

প্রশ্ন। একটি প্রোগ্রাম লেখো যা ব্যবহারকারীর কাছে **0-এর বড় দুটি সংখ্যা** চায়, `C` ও `R`। প্রোগ্রাম তারকাচিহ্ন দিয়ে একটি **আয়তক্ষেত্র** ছাপায়, **প্রস্থে** `C` আর **উচ্চতায়** `R`।
উদাহরণ: `C` 5 আর `R` 3 হলে প্রোগ্রাম ছাপায়

```
*****
*****
*****
```

নামগুলো মনে রাখতে সাহায্য করে: `C` মানে **colonne** (কলাম, প্রস্থ), `R` মানে **righe** (সারি, উচ্চতা)।

3.1 দুটি প্রশ্ন

প্রথমে দুটি ভেরিয়েবল ঘোষণা করি, `int C` ও `int R`। তারপর দুটি প্রশ্ন, প্রতিটি নিজের **do while**-সহ (সংখ্যা 1-এর ছোট হলে পুনরাবৃত্তি), ঠিক অনুশীলন 1-এর মতো।

সতর্কতা

প্রশ্নের **প্রসঙ্গ দাও**। প্রথম সংখ্যা আয়তক্ষেত্রের **প্রস্থ** (**larghezza**): তাই `printf` "প্রথম সংখ্যা দাও" বলে না, বলে `"Inserisci la larghezza del rettangolo"`। দ্বিতীয়টি: `"Inserisci l'altezza del rettangolo"`। ব্যবহারকারীকে বুঝতে হবে তুমি **কী** চাইছ।

ডেটা চাওয়ার সময় `for` বসানো হয় না: তখনো জানা নেই **কতবার** পুনরাবৃত্তি হবে। দুটি ডেটা নেওয়ার **পরে** লেখা হয়।

3.2 সমস্যাকে দুই ভাগ করা

আয়তক্ষেত্র ছাপানো অনুশীলন 1-এর চেয়ে একটু জটিল। **সমস্যাকে** দুই অংশে **ভাগ করা** ভালো।

1. **প্রথম অংশ:** `C`টি তারকাচিহ্নের **এক সারি** ছাপানো। এটি আমরা জানি: অনুশীলন 1, একটি `for` যেখানে `x` 0 থেকে `x < C` পর্যন্ত, আর ভেতরে `printf("*");`।
2. **দ্বিতীয় অংশ:** এই সারি `R` **বার** ছাপাতে হবে (উদাহরণে 3)। কপি-পেস্টে একই `for` তিনবার লিখতে পারি, কিন্তু `R` 50 হলে? **এভাবে করা হয় না**: পুরো `for` ব্লক **আরেকটি `for`-এর ভেতরে** রাখা হয়, যা সেটিকে `R` বার পুনরাবৃত্তি করে। একটি **আলাদা ভেরিয়েবল** লাগে: `y`, যা 0 থেকে `y < R` পর্যন্ত।

এভাবে প্রোগ্রাম এমন কিছু **3 বার** পুনরাবৃত্তি করে যা **5 বার** পুনরাবৃত্তি করে। `if`-এর ভেতরে `if`-কে কী বলে মনে আছে? **নেস্টেড** (**annidato**)। একইভাবে `for`-এর ভেতরে `for`-কে বলে **নেস্টেড for** (বা নেস্টেড লুপ)। `y`-সহ `for` হলো **বাইরের** লুপ (সারি), `x`-সহটি **ভেতরের** লুপ (কলাম)।

3.3 প্রথম চেষ্টা: কিছু বাকি

শুধু দুটি `for`, একটির ভেতরে আরেকটি, ভেতরেরটিতে `printf("*");` লিখলে কোড **ঠিকভাবে লেখা** (ভুল নেই), কিন্তু আমরা যা চাই তা **এখনো করে না**: 5 ও 3 দিলে আয়তক্ষেত্র আসে না, আসে 15টি তারকাচিহ্নের **একটিমাত্র সারি**।

```
***************
```

কী বাকি? প্রতিবার **একটি সারি** লেখা শেষে **নতুন লাইনে যেতে** হবে। প্রতি তারকাচিহ্নে নয়, **প্রতি 5টি তারকাচিহ্নে**, অর্থাৎ ভেতরের লুপ শেষ হলে। তাই `printf("\n");` যায় **ভেতরের for-এর পরে**, কিন্তু **বাইরের for-এর ভেতরে**।

```c
#include <stdio.h>

int main()
{
    int C;
    int R;

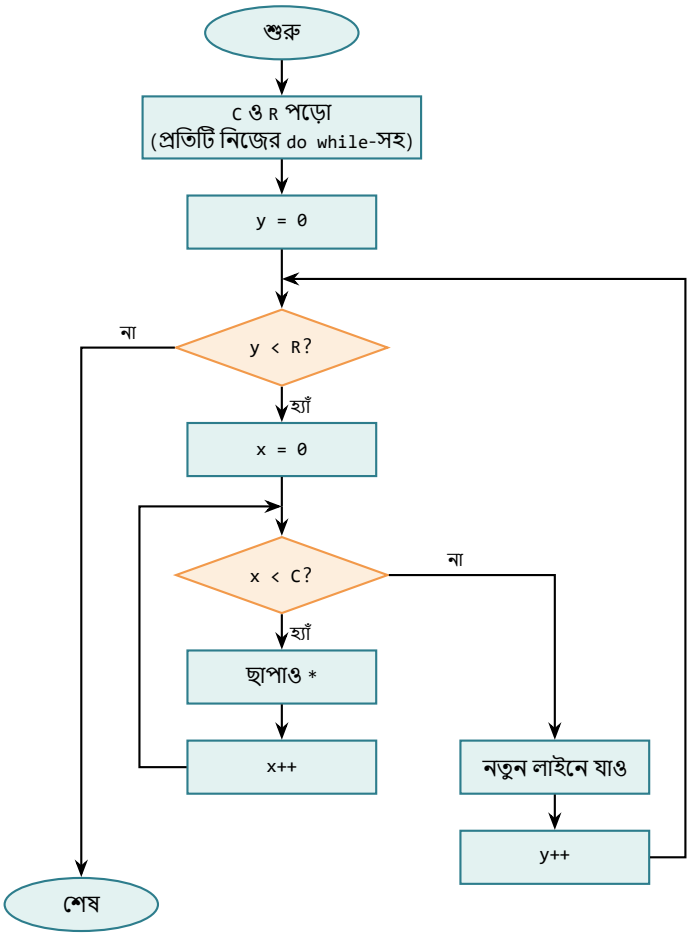
    // আয়তক্ষেত্রের প্রস্থ (কলাম)
    do
    {
        printf("Inserisci la larghezza del rettangolo: ");
        scanf("%d", &C);
    } while (C < 1);

    // আয়তক্ষেত্রের উচ্চতা (সারি)
    do
    {
        printf("Inserisci l'altezza del rettangolo: ");
        scanf("%d", &R);
    } while (R < 1);

    // বাইরের লুপ: প্রতি সারিতে একবার
    for (int y = 0; y < R; y++)
    {
        // ভেতরের লুপ: C টি তারকাচিহ্নের এক সারি
        for (int x = 0; x < C; x++)
        {
            printf("*");
        }
        // সারি শেষ, নতুন লাইন
        printf("\n");
    }

    return 0;
}
```

```
Inserisci la larghezza del rettangolo: 5
Inserisci l'altezza del rettangolo: 3
*****
*****
*****
```



ছবি/নকশা 2: অনুশীলন 2: দুটি নেস্টেড লুপ। ভেতরের লুপ (x) এক সারি ছাপায়; বাইরের লুপ (y) সেটিকে R বার পুনরাবৃত্তি করে, প্রতি সারির পরে নতুন লাইনে গিয়ে।

4 অনুশীলন 3: ফাঁপা আয়তক্ষেত্র

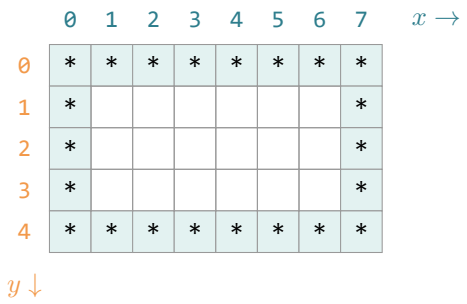
ভরা আয়তক্ষেত্রের পরে ক্লাস থেকে একটি অনুরোধ এল: **যদি** **ফাঁপা** **চাই**, অর্থাৎ তারকাচিহ্ন **শুধু** **সীমানায়**? এটিই সঠিক পথ: প্রতিদিন আমরা **আরও** **জটিল** কোড করব। চেষ্টা করি।

উদাহরণ

প্রশ্ন। অনুশীলন 2-এর প্রোগ্রাম বদলাও (ক্লাসে একই প্রোগ্রামে কাজ হয়েছে), যাতে আয়তক্ষেত্র **ফাঁপা** হয়: তারকাচিহ্ন ছাপা হয় **শুধু** **সীমানায়**, ভেতরে **ফাঁক**।

4.1 স্থানাঙ্ক, battaglia navale-র মতো

একটি বড় আয়তক্ষেত্র আঁকি, **8** **বাই** **5** (`C = 8`, `R = 5`), এবং প্রতিটি কলাম ও সারিকে একটি নম্বর দিই, **battaglia** **navale** খেলার মতো। কলাম হলো `x`, ডানে 0 থেকে 7; সারি হলো `y`, নিচে 0 থেকে 4। **0** থেকে শুরু, কারণ আমাদের `for` 0 থেকে শুরু হয়।



তারকাচিহ্ন কখন ছাপা হয়?

- `y` **0** হলে: পুরো **প্রথম** **সারি** ছাপা হয়;
- `x` **0** হলে: পুরো **প্রথম** **কলাম** ছাপা হয়;
- **শেষ** **সারিতে**। শেষ সারিতে `y` কত? আঁকায় 4, কিন্তু **সবসময়** **নয়**: উচ্চতা 5 হলে শেষ সারি 4, উচ্চতা 10 হলে 9, 18 হলে 17। সাধারণভাবে শেষ সারি **`R - 1`**, কারণ গোনা 0 থেকে শুরু;
- **শেষ** **কলামে**, একই কারণে: **`x == C - 1`**।

তাই ভেতরের লুপে তারকাচিহ্নের `printf` একটি `if`-এর নিচে যায়: ছাপে যদি `x == 0` **অথবা** `y == 0` **অথবা** `y == R - 1` **অথবা** `x == C - 1`। "অথবা" হলো **OR** অপারেটর, `||`।

4.2 একবারে একটি শর্ত

প্রথম দুটি শর্তে (`x == 0 || y == 0`) প্রোগ্রাম সত্যিই প্রথম সারি ও প্রথম কলাম ছাপে। শেষ সারি যোগ করি, `y == (R - 1)`, আর 8 ও 5 দিয়ে দেখি:

```
********
*
*
```

```
*
********
```

নিচের সারিও এল। কী বাকি? **শেষ কলাম**: `x == (C - 1)`।

4.3 else ছাড়া চেষ্টা: তারকাচিহ্ন গাদা হয়ে যায়

শেষ কলাম যোগ করলেও ফলাফল **কাঙ্ক্ষিত নয়**: যুক্তি নিখুঁত, যন্ত্র **ঠিক যা লিখেছি তাই** করেছে, কিন্তু শেষ কলাম তার জায়গায় **নেই**: তারকাচিহ্নগুলো প্রথম কলামের সাথে **জুড়ে** গেছে।

```
********
**
**
**
********
```

সমস্যা হলো **সব শর্ত মিথ্যা** হলে কী করতে হবে তা বলতে ভুলে গেছি। কিছু না বললে প্রোগ্রাম **কিছু ছাপে না**: প্রথম কলামের তারকাচিহ্ন ছাপে, আর `x` 7 হলে দ্বিতীয়টি **ঠিক পাশে** ছাপে। অথচ প্রতিবার শর্ত পূরণ না হলে একটি **ফাঁক** ছাপতে হবে: ফাঁক দেখা যায় না, কিন্তু **জায়গা নেয়** এবং শেষ কলামের তারকাচিহ্নকে ঠিক জায়গায় পৌঁছাতে দেয়। একটি ফাঁক ছাপানো **else** যোগ করলেই হয়।

```
    for (int y = 0; y < R; y++)
    {
        for (int x = 0; x < C; x++)
        {
            // শুধু সীমানায় তারকাচিহ্ন, নাহলে একটি ফাঁক
            if (x == 0 || y == 0 || y == (R - 1) || x == (C - 1))
            {
                printf("*");
            }
            else
            {
                printf(" ");
            }
        }
        printf("\n");
    }
```

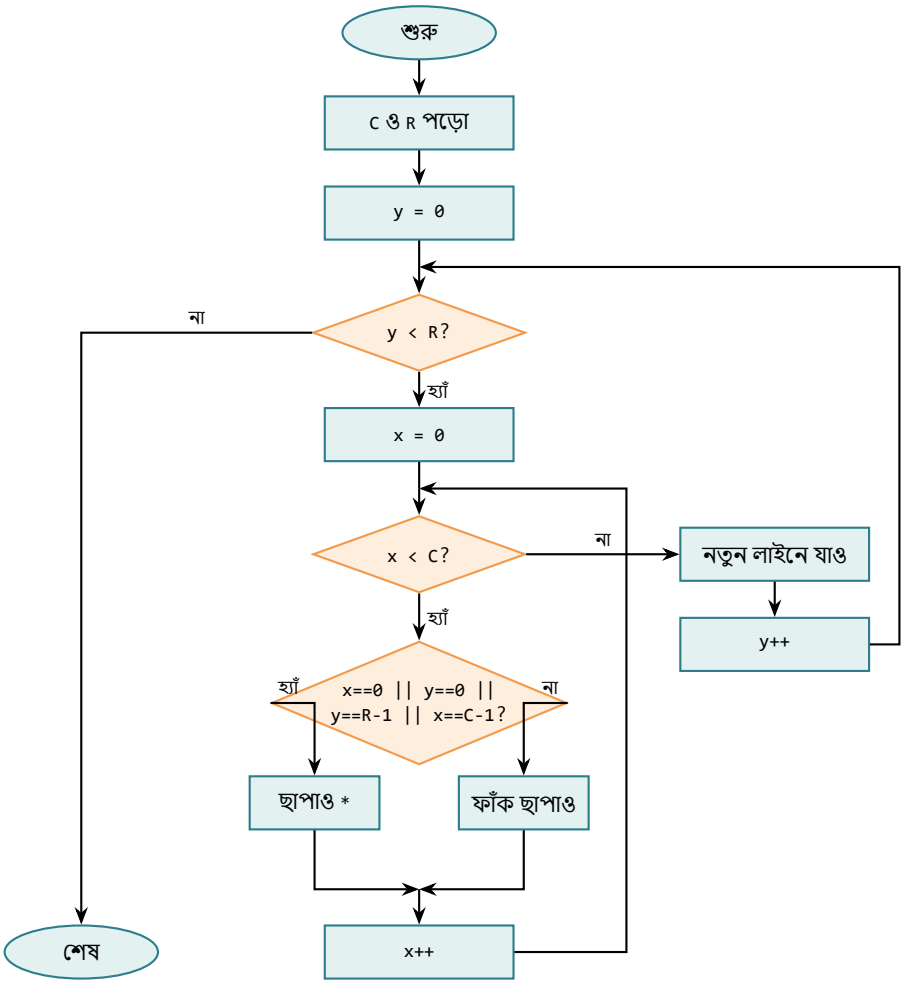
`R - 1` ও `C - 1`-এর চারপাশের বন্ধনী বাধ্যতামূলক নয় (`==` তুলনা বিয়োগের পরে হয়), কিন্তু শর্তকে **পড়তে সহজ** করে: ক্লাসে আমরা দিয়েছি।

```
Inserisci la larghezza del rettangolo: 8
Inserisci l'altezza del rettangolo: 5
********
*      *
*      *
*      *
********
```

টীকা

ফাঁক দেখার জন্য। ফাঁকের বদলে একটি **বিন্দু** ছাপিয়ে দেখো, `printf(".");`: আয়তক্ষেত্রের মাঝে বিন্দু ছাপা হয়। ওই বিন্দুগুলোই **খালি জায়গা** যা আমরা ভরতে চাই না: তাই শেষ রূপে সেখানে একটি ফাঁক থাকে।

```
********
*......*
*......*
*......*
********
```



ছবি/নকশা 3: অনুশীলন 3: ভেতরের লুপে একটি `if ... else` তারকাচিহ্ন (সীমানা) আর ফাঁকের (ভেতর) মধ্যে বাছে।

4.4 ইন্ডেন্টেশন ভুল খুঁজতে কাজে লাগে

এই অনুশীলনে বন্ধনী অনেক: দুটি `for`, একটি `if`, একটি `else`। কোড ইন্ডেন্টেশন ছাড়া **সব সোজা** লেখা হলে কিছুই বোঝা যায় না, আর ভুল খোঁজা (একটি বন্ধনী নেই, একটি সেমিকোলন বেশি) খুব কঠিন হয়, **শিক্ষকের জন্যও**। নিয়ম:

- প্রতিবার **বন্ধনী খুললে** ভেতরের কোড **এক ট্যাব** ডানে সরে;
- **বন্ধনী** যে শব্দ তা খোলে তার **নিচে** থাকে: `for`-এর `f`-এর নিচে, `if`-এর `i`-এর নিচে;
- দুটি `for`-এর ভেতরের একটি `if`-এর ভেতরের `printf` **তিন ট্যাব** সরে (`main`-এরটি ছাড়াও)।

onlinegdb-র এডিটর **খাড়া রেখা** আঁকে, যা দেখায় একটি ব্লক কোথায় **শুরু ও শেষ**: ইন্ডেন্ট করা কোডে সঙ্গে সঙ্গে দেখা যায় বন্ধনী আছে কিনা। এভাবে পাওয়া সাধারণ ভুল: `for` খোলার বন্ধনী নেই; `printf("\n");` **ভুল জায়গায়** (ভেতরের লুপের পরে যাবে, ভেতরে নয়); `return 0;` শেষ বন্ধনীর আগে না হয়ে **পরে**; `for` বা `if`-এর বন্ধনীর পরে সেমিকোলন, যা লুপ খালি করে দেয়; `y`-এর জায়গায় `x` লেখা।

> **টীকা**
>
> অক্ষর দিয়ে এই আঁকাগুলো **গ্রাফিক্সের** ভিত্তি: আগামী বছর তারকাচিহ্নের বদলে আমরা **পিক্সেল** জ্বালাব, আর সেগুলো সরিয়ে অ্যানিমেশন ও **Pong**-এর মতো সহজ ভিডিও গেম করা যায় (দুটি র‍্যাকেট ও বল)। Pac-Man-এর মতো গেম বেশি জটিল, বিশেষ করে গ্রাফিক্সে।

5 অনুশীলন 4: সমকোণী ত্রিভুজ

উদাহরণ

প্রশ্ন। একটি প্রোগ্রাম লেখো যা ব্যবহারকারীর কাছে **0-এর বড়** এবং **10-এর ছোট** একটি সংখ্যা `N` চায়। প্রোগ্রাম `N` বাহুর তারকাচিহ্নের একটি **সমকোণী ত্রিভুজ** ছাপায়।
উদাহরণ: `N` 5 হলে প্রোগ্রাম ছাপায়

```
*
**
***
****
*****
```

সমকোণী ত্রিভুজ হলো যে ত্রিভুজের একটি **90 ডিগ্রির কোণ** আছে। এটি **সমদ্বিবাহুও**: দুটি বাহু সমান, দৈর্ঘ্য `N`।

সতর্কতা

যা চাওয়া হয়েছে তা মানো। প্রোগ্রামকে **প্রশ্নে যা চাওয়া তাই** করতে হবে: ব্যবহারকারীর কাছে একটি সংখ্যা চেয়ে সেই বাহুর ত্রিভুজ ছাপাতে হবে। অনেক `printf` দিয়ে একটি স্থির ত্রিভুজ ছাপানো প্রোগ্রাম, হয়তো ইন্টারনেট থেকে কপি করা, সমাধানের **আরেকটি উপায় নয়**: এটি **অন্য জিনিস**। পিৎজার দোকানে margherita চেয়ে স্যামনের টুকরো পেলে, আর একই দাম দিতে হলে, তুমি মেনে নেবে? উদাহরণ থেকে ধারণা নেওয়া চলে, কিন্তু যা চাওয়া হয়েছে তার সাথে **মানিয়ে** নিতে হবে।

5.1 প্রথম সমস্যা: 1 থেকে 9 সংখ্যা

"0-এর বড় এবং 10-এর ছোট" মানে অনুমোদিত সংখ্যা **1 থেকে 9**: 0 চলে না, 10-ও না।

`do while`-এর শর্ত বলে **কখন পুনরাবৃত্তি**, ডেটা কখন ঠিক তা নয়। যুক্তি দুই ধাপে:

1. লিখি সংখ্যা কখন **ঠিক**: `N > 0 && N < 10`। এটি "0-এর বড় **এবং** 10-এর ছোট": "এবং" হলো **AND** অপারেটর, `&&`;
2. কিন্তু আমাদের পুনরাবৃত্তি করতে হবে সংখ্যা ঠিক **না** হলে: পুরো শর্ত বন্ধনীতে রেখে সামনে **বিস্ময়চিহ্ন** দিই, **NOT**, যা **উল্টে দেয়**: সত্যকে মিথ্যা করে, মিথ্যাকে সত্য।

```c
    do
    {
        printf("Inserisci il numero: ");
        scanf("%d", &N);
    } while ( !((N > 0) && (N < 10)) );
    // সংখ্যা 0-এর বড় ও 10-এর ছোট না হলে পুনরাবৃত্তি
```

সতর্কতা

সাধারণ ভুল: AND-এর বদলে OR। `N > 0 || N < 10` ("0-এর বড় **অথবা** 10-এর ছোট") লেখা ভুল: **প্রতিটি** সংখ্যা 0-এর বড় অথবা 10-এর ছোট (−5-ও, 42-ও), তাই এই শর্ত **সবসময়** **সত্য**। OR দিলে প্রোগ্রাম **চাইতেই** **থাকে** বা সব মেনে নেয়: প্রশ্নে যা চাওয়া তা করে না। মনে রেখো: "অথবা" হলো **OR**, "এবং" হলো **AND**। আর যে অপারেটর **উল্টো** করে, সত্যকে মিথ্যা ও মিথ্যাকে সত্য, তার নাম কী? NOT। বিস্ময়চিহ্ন "NOR" **নয়**: এটি **NOT**।

এগোনোর আগে পরীক্ষা করি (**Inserisci il numero** = সংখ্যা দাও):

```
Inserisci il numero: -5
Inserisci il numero: 0
Inserisci il numero: 15
Inserisci il numero: 42
Inserisci il numero: 5
```

−5, 0, 15 ও 42 দিলে প্রশ্ন আবার হয়; 5 দিলে প্রোগ্রাম এগোয়। 1 থেকে 9-এর অন্য সংখ্যাগুলোও দেখা যায়। প্রথম সমস্যা, ডেটা নেওয়া, মিটে গেছে।

টীকা

একই শর্ত NOT ছাড়াও লেখা যায়, সরাসরি বলে সংখ্যা কখন ভুল: `N < 1 || N > 9` (1-এর ছোট **অথবা** 9-এর বড়)। দুটি লেখা **সমতুল্য**।

5.2 একটি নিয়ম: কয় মাত্রা, কয় for

সংজ্ঞা

কিছু আঁকার সময়, আঁকার **মাত্রার** **সংখ্যা** **নেস্টেড** **for-এর** **সংখ্যার** সমান:

- **একটি** **রেখায়** আঁকা (1 মাত্রা), অনুশীলন 1-এর সারির মতো: **একটি** **for**;
- **তলে** আঁকা (2 মাত্রা): বর্গ, ত্রিভুজ, বৃত্ত, গাছসহ ছোট বাড়ি: **দুটি** **for**, একটি অন্যটির ভেতরে;
- **বাস্তব** **জগতের** বস্তু (3 মাত্রা): **তিনটি** **for**, একটি `for`-এর ভেতরে `for`, তার ভেতরে আরেকটি `for`।

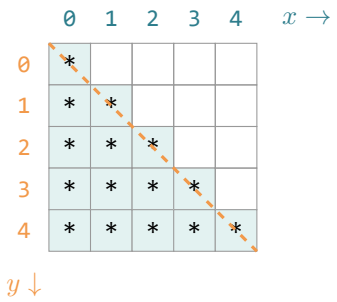
এটি কোনো কৌশল নয়: এটি সব আঁকার জন্য সত্য একটি **কথা**।

ত্রিভুজ 2 মাত্রার আঁকা: **দুটি** **নেস্টেড** **for** লাগে, আয়তক্ষেত্রের একই গঠনে, **ভেতরের** **for-এর** শেষে `printf("\n");` সহ।

5.3 বর্গ থেকে ত্রিভুজ

প্রথম ধাপ হলো ভরা বর্গ: দুটি `for`, দুটোই `N` পর্যন্ত। 5 দিলে প্রোগ্রাম 5 বাই 5 তারকাচিহ্নের বর্গ ছাপে। এখনো ত্রিভুজ নয়, তবে কাছাকাছি।

এবার স্থানাঙ্ক দেখি, ফাঁপা আয়তক্ষেত্রের মতো: কলাম `x` 0 থেকে 4, সারি `y` 0 থেকে 4।



তারকাচিহ্ন কখন ছাপা হয়? যখন আমরা **কর্ণের নিচে** (বা তার ওপরে) থাকি। `y = 2` সারি নিই: তারকাচিহ্ন 3টি, কলাম 0, 1 ও 2-তে। কর্ণের বিন্দুর `x = 2` ও `y = 2`। ভেতরের লুপে আমরা আনুভূমিকভাবে যাই বলে, `y` সারিতে তারকাচিহ্ন ছাপা হয় যতক্ষণ `x`, `y`-এর **ছোট বা সমান**। তাই ভেতরের লুপ **`N` পর্যন্ত যায় না**, `x <= y`-তে থামে: শুধু ভেতরের `for`-এর শর্ত বদলায়।

```c
#include <stdio.h>

int main()
{
    int N;

    // সংখ্যা 0-এর বড় ও 10-এর ছোট না হলে পুনরাবৃত্তি
    do
    {
        printf("Inserisci il numero: ");
        scanf("%d", &N);
    } while (!(N > 0 && N < 10));

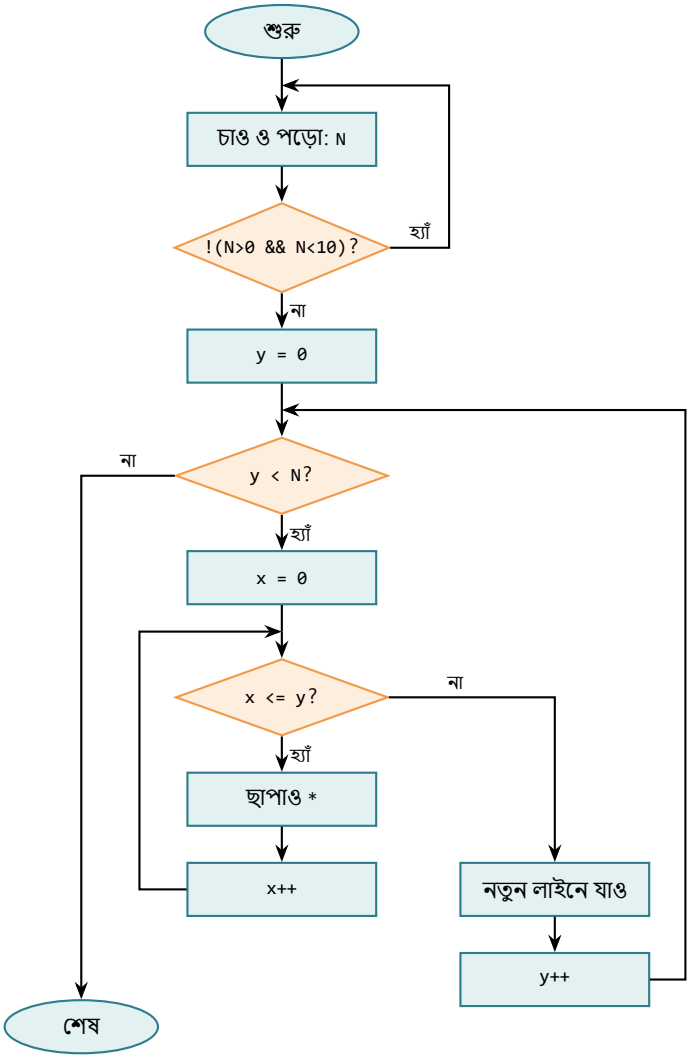
    // 2 মাত্রার আঁকা: দুটি নেস্টেড for
    for (int y = 0; y < N; y++)
    {
        // y সারিতে x <= y যতক্ষণ তারকাচিহ্ন ছাপাই
        for (int x = 0; x <= y; x++)
        {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

```
Inserisci il numero: 5
*
**
***
****
*****
```

টীকা

এটি পাঠের শেষ কয়েক মিনিটে ক্লাসে লেখা প্রোগ্রাম, 5 দিয়ে পরীক্ষা করা। দেখার আগে নিজে ভরা বর্গ থেকে শুরু করে আবার তৈরি করার চেষ্টা করো।



ছবি/নকশা 4: অনুশীলন 4: NOT দিয়ে ডেটা যাচাই, তারপর দুটি নেস্টেড লুপ। আয়তক্ষেত্রের তুলনায় শুধু ভেতরের লুপের শর্ত বদলায়: `x <= y`।

6 এই পাঠ থেকে যা মনে রাখতে হবে

- **ডেটা চাইতে** এবং ভুল হলে প্রশ্ন আবার করতে: **do while**, শর্তে **কখন পুনরাবৃত্তি** (ভুল ডেটা)। "0-এর বড়" ⇒ `N < 1` হলে পুনরাবৃত্তি।
- একটি ডেটা যৌগিক শর্ত **না** মানলে পুনরাবৃত্তি করতে: সঠিক শর্ত **AND** (`&&`) দিয়ে লিখে **NOT** (`!`) দিয়ে উল্টে দাও। "এবং" হলো AND, "অথবা" হলো OR।
- প্রতিটি ডেটা টাইপের একটি **সীমা** আছে: 32 বিটের `int` প্রায় -2 থেকে $+2$ বিলিয়ন; তার বাইরে **overflow**।
- কঠিন সমস্যাকে **অংশে ভাগ করা** হয়: আগে এক সারি, তারপর সারির পুনরাবৃত্তি।
- `for`-এর ভেতরে `for` হলো **নেস্টেড for**: **বাইরের**টি সারি গোনে (`y`), **ভেতরের**টি কলাম (`x`), **দুটি আলাদা ভেরিয়েবল** দিয়ে। **নতুন লাইন** ভেতরের লুপের পরে।
- আঁকার **মাত্রা** = **নেস্টেড for-এর** সংখ্যা।
- প্রতিটি বিন্দুতে **কী ছাপাবে** বাছতে **স্থানাঙ্ক** `x` ও `y` ব্যবহার হয়, 0 থেকে শুরু: **শেষ** সারি `R - 1`, **শেষ** কলাম `C - 1`। অক্ষর না ছাপালে **ফাঁকসহ** একটি **else** লাগে।
- **ইন্ডেন্ট করা** কোডে ভুল সঙ্গে সঙ্গে পাওয়া যায়; প্রোগ্রাম **প্রতি ধাপে** পরীক্ষা করা হয়।
- প্রোগ্রামকে **প্রশ্নে যা চাওয়া তাই** করতে হবে।

> **টীকা**
>
> **বাড়ির কাজ।** Classroom-এ পাবে **কাজ C01**: নেস্টেড লুপে আরও আকৃতি (উল্টো ত্রিভুজ, সংখ্যার ত্রিভুজ, ডানে ত্রিভুজ, X), `while` দিয়ে উল্টো গণনা আর `switch` দিয়ে একটি মেনু। ক্লাসে যেখানে থেমেছি সেখান থেকে চলবে।

7 শব্দকোষ: ক্লাসে ব্যবহৃত ইতালীয় শব্দ

- **asterisco**: তারকাচিহ্ন
- **riga / colonna**: সারি / কলাম
- **larghezza / altezza**: প্রস্থ / উচ্চতা
- **rettangolo pieno / vuoto**: ভরা / ফাঁপা আয়তক্ষেত্র
- **bordo**: সীমানা
- **spazio**: ফাঁক
- **annidato**: নেস্টেড
- **ciclo esterno / interno**: বাইরের / ভেতরের লুপ
- **triangolo rettangolo**: সমকোণী ত্রিভুজ
- **diagonale**: কর্ণ

- **a capo**: নতুন লাইনে
- **battaglia navale**: কাগজে খেলার একটি যুদ্ধজাহাজ খেলা